



Cryptography

Nicola Cibir, Dr. Alex Stefanov

MSc EE & MSc SET, Course EE4410

Faculty of Electrical Engineering, Mathematics and Computer Science, **TU Delft**

Lecture 3 on May 6th, 2025

Disclaimer & Ethics

1. **All information provided in the course is for educational purposes only.** The lecturer(s) and teaching assistant(s) are not responsible for any misuse of the information.
2. The course does not promote and is strongly against illegal activities such as unethical hacking, cracking and piracy.
3. **Do not attempt to violate the law with anything contained in the course.** Do not abuse the material provided and be responsible.
4. By participating in the course, **you (student) must accept that you will only lawfully use the material and knowledge you got from this course.**
5. Any actions and / or activities related to the material contained within this course are solely your (student) responsibility. **The misuse of the information in this course can result in criminal charges brought against the persons in question.**
6. The lecturer(s) and teaching assistant(s) will not be held responsible in the event any criminal charges are brought against any individuals misusing the information from this course to break the law.

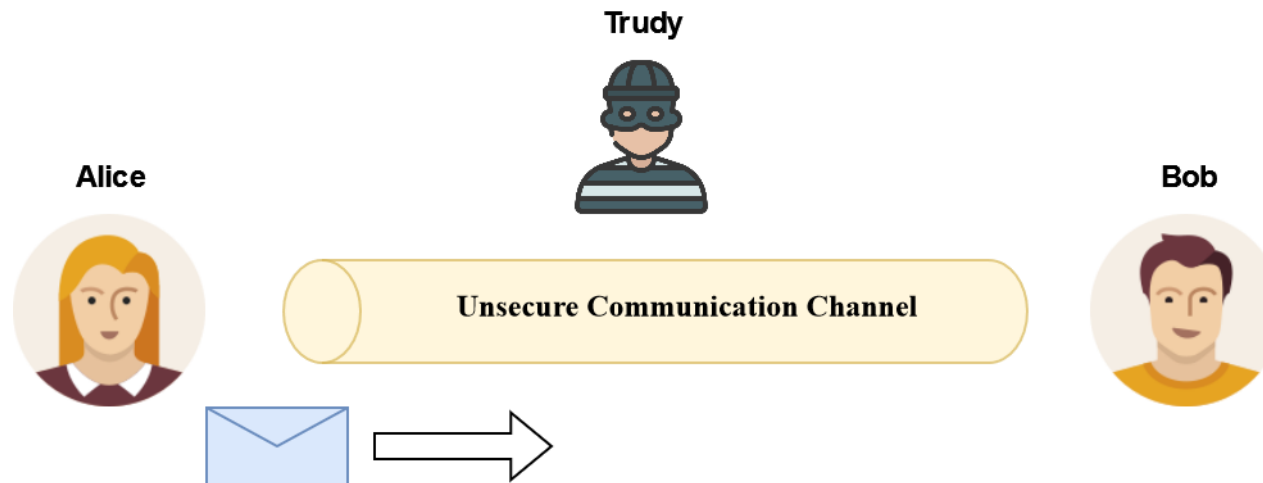
Agenda

- Introduction
- Symmetric Encryption
 - Ciphers
 - Operation Modes
 - Message Authentication Code
- Hashing
 - Hash Function
 - Cryptographic Hash Function
- Asymmetric Encryption
 - Rivest, Shamir, Adleman (RSA)
 - Elliptic Curves Cryptography (ECC)
- Digital Signatures

Introduction

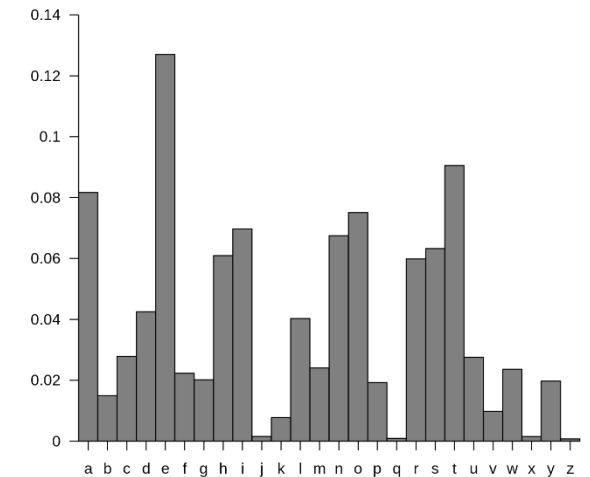
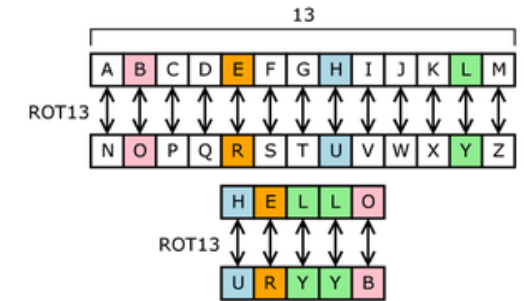
“**Cryptography** is the practice and study of techniques for secure communication in the presence of adversarial behavior”

- How to communicate **confidential** information over an **unsecure** channel?



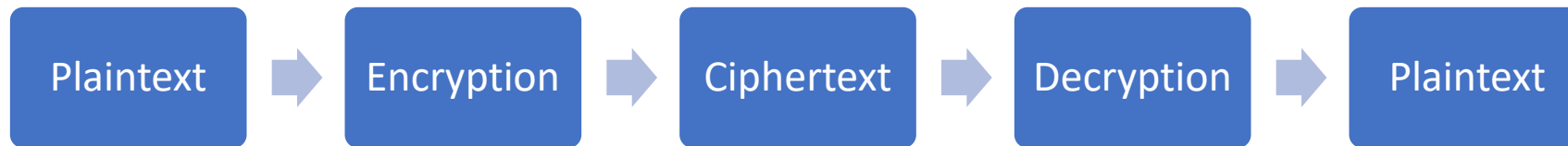
Introduction

- Some **historical** encryption techniques:
 - Ancient Greek Scitale (7th century B.C.)
 - Transposition cipher
 - Cesar Cipher (58 B.C, Julius Cesar)
 - Bruteforce attack: try every right-side translation
 - Substitution Cipher
 - Symbol frequency as attack knowledge
 - One-Time Pad
 - Proven as unbreakable by Claude Shannon during WW2
 - Impractical: key as long as plaintext, random, used once and **only** once
 - $\sum_{i=0}^n c_i = m_i \oplus k_i$



Introduction

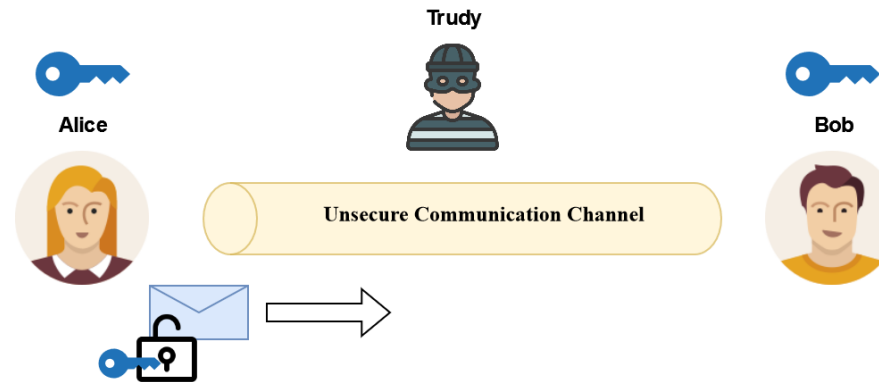
- **Encryption** is the central function of cryptography
 - Transformation of data into an unreadable form
 - Encryption ensures privacy by keeping the information hidden from those for whom the information is not intended
- **Decryption**, opposite of encryption, transforms encrypted data back into an intelligible form
- Encryption/decryption process involves
 - Taking data in plaintext, which is readable and understandable text
 - Manipulating its form to create ciphertext
 - When data has been transformed into ciphertext, the plaintext becomes inaccessible until it's decrypted



Symmetric Encryption

- The **same** key is used for plaintext encryption and ciphertext decryption
 - Each user must possess the shared key to send encrypted messages and decrypt them
 - The sender uses the key to encrypt the message and then transmits it to the receiver
 - The receiver in possession of the same key, uses it to decrypt the message

$$\forall k \in \mathbb{K}, \forall m \in \mathbb{P}, d_k(e_k(m)) = m$$



- The security of symmetric encryption relies on the **secrecy of the shared secret key**

Symmetric Encryption - Ciphers

- Stream Ciphers

- Encrypt the digits (typically bytes), or letters (in substitution ciphers) of a message **one at a time**
- By design, they are fast compared to block ciphers
- Substitution ciphers are well-known stream ciphers → vulnerable to frequency table attacks

- Block Ciphers

- Operate on **blocks of plaintext** one at a time to produce **blocks of ciphertext**
- The block of plaintext and the block of ciphertext are assumed to be of the same size
 - ✓ Padding the plaintext to achieve a multiple of the block size

Symmetric Encryption – RC4

- Ron's Cipher, after Ron Rivest from MIT
- Very, very fast **stream cipher**
- Designed in 1987 and use in TLS until 2015
- Algorithm consists of:
 - Key Scheduling Algorithm (KSA)
 - Pseudo-Random Generation Algorithm (PRGA)
 - RC4 output is a keystream of bytes **K** which is XORed with the plaintext byte by byte
- Shown that the random stream produced by the RC4 algorithm does not behave in a random manner
 - As of 2015, IETF prohibits the use of RC4 in TLS. Mozilla and Microsoft issued similar recommendations

Symmetric Encryption – RC4

Implementation details not required for the exam!

```
def KSA(key):
    """
    Key Scheduling Algorithm (KSA)
    """
    key_length = len(key)
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % key_length]) % 256
        S[i], S[j] = S[j], S[i]
    return S

def PRGA(S):
    """
    Pseudo-Random Generation Algorithm (PRGA)
    """
    i = 0
    j = 0
    while True:
        # Makes sure every array element is used once after 256 iterations
        i = (i + 1) % 256
        # Makes the output depend non-linearly on the array
        j = (j + S[i]) % 256
        # Makes sure the array is evolved and modified as the iteration continues
        S[i], S[j] = S[j], S[i]
        # Makes sure the output sequence reveals little about the internal state of the array
        K = S[(S[i] + S[j]) % 256]
        yield K
```

```
def RC4(key):
    """
    RC4 encryption algorithm
    """
    S = KSA(key)
    keystream = PRGA(S)
    return keystream

def encrypt(plaintext, key):
    """
    Encrypts plaintext using RC4 algorithm with given key
    """
    keystream = RC4(key)
    encrypted = bytearray()
    for char in plaintext:
        encrypted.append(char ^ next(keystream))
    return encrypted

def decrypt(ciphertext, key):
    """
    Decrypts ciphertext using RC4 algorithm with given key
    """
    return encrypt(ciphertext, key) # RC4 is symmetric
```

Symmetric Encryption – DES

- Data Encryption Standard (DES)
- Introduced in 1972 by a team in IBM which included Horst Feistel
- Some of the design has been amended by NSA to make the cipher more secure
- DES is a **block cipher** based on the basic **Feistel cipher**

- **Feistel cipher**

- The interesting property of a **Feistel cipher** is that the **round function** is **invertible** regardless of the choice of the function **F**

Encryption

$$L_i \leftarrow R_{i-1}$$

$$R_i \leftarrow L_{i-1} \oplus F(K_i, R_{i-1})$$

Decryption

$$R_{i-1} \leftarrow L_i$$

$$L_{i-1} \leftarrow R_i \oplus F(K_i, L_i)$$

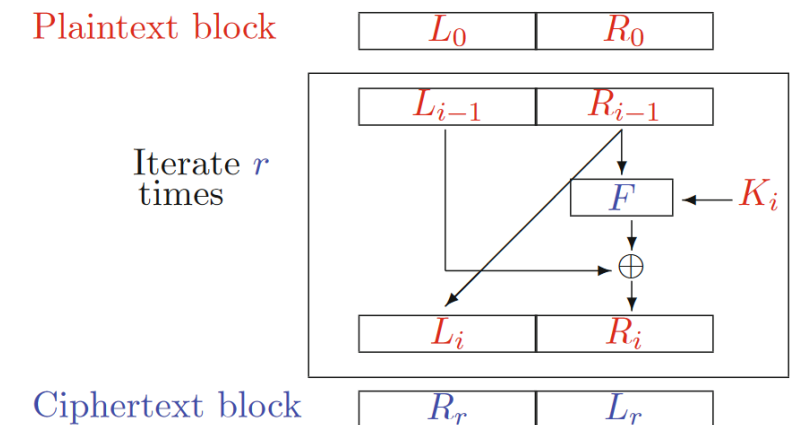


FIGURE 13.2. Basic operation of a Feistel cipher

Symmetric Encryption – DES

- **Feistel cipher** has simplified the design of a cipher, since:
 - Any function **F** can be chosen. Still, the encryption function can be inverted using the secret key
 - The same code/circuitry can be used for encryption and decryption functions
Only need to use the round keys in reverse order for decryption
- To obtain a secure cipher, we still need to take care of:
 - How the **round keys** are generated
 - How many **rounds** to take
 - How the round function **F** is defined

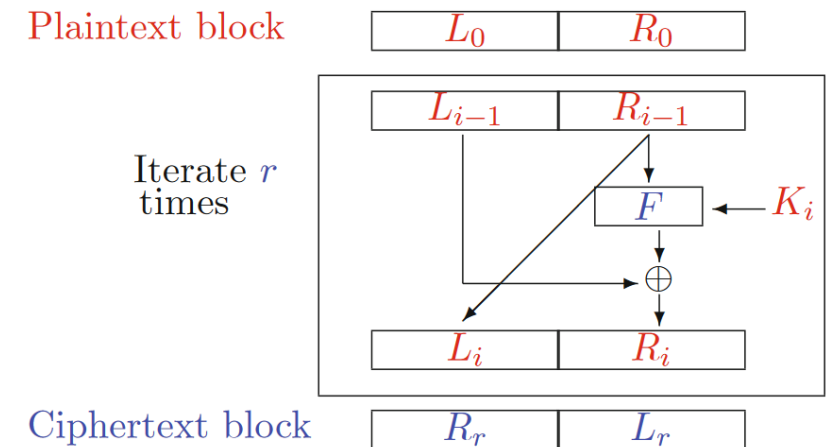
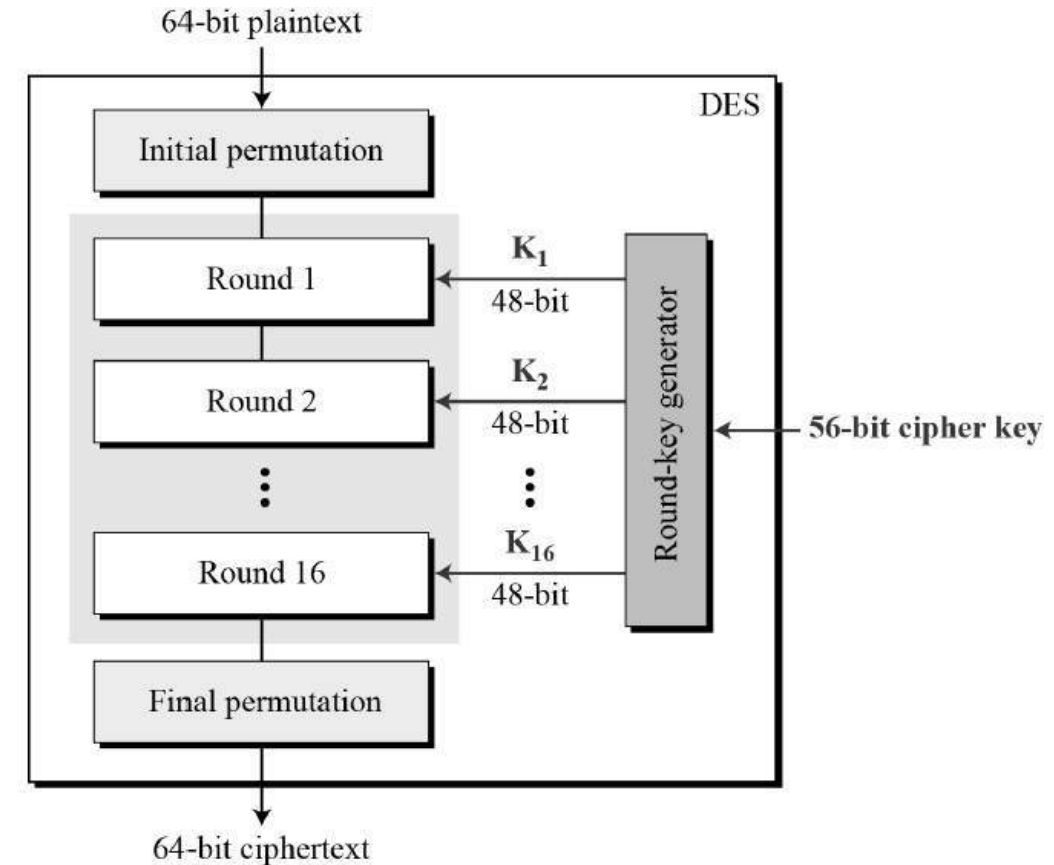


FIGURE 13.2. Basic operation of a Feistel cipher

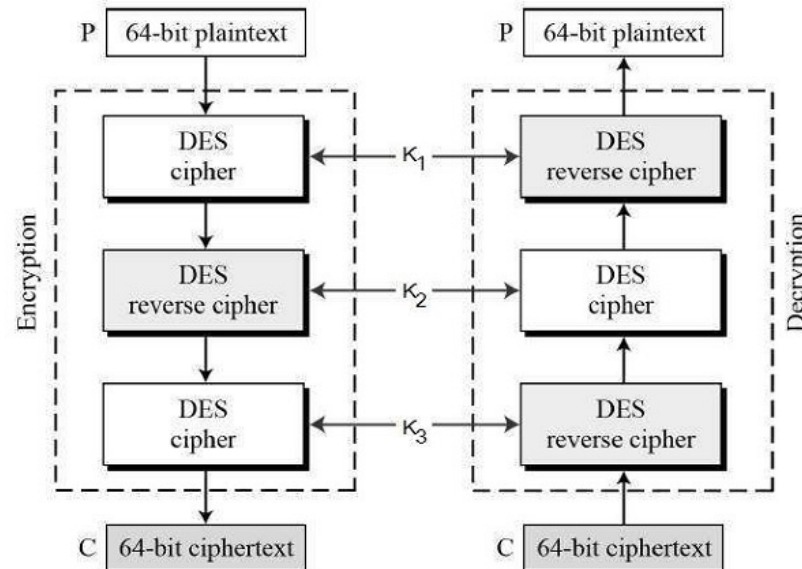
Symmetric Encryption – DES

- DES implementation
- It is a variant of the Feistel cipher design in which:
 - The number of rounds r is 16
 - The block length n is 64 bits
 - The key length is 56 bits
 - The round keys $K_1, \dots, K_r$ are each 48 bits long
- DES stood up well to the test of time, but in the early 1990s it became clear that a new standard was required
- Block length and key length of basic DES were too small
 - In 1998, brute-force attack, 56 hours
 - From 2012, 26 hours with specialized hardware



Symmetric Encryption – 3DES

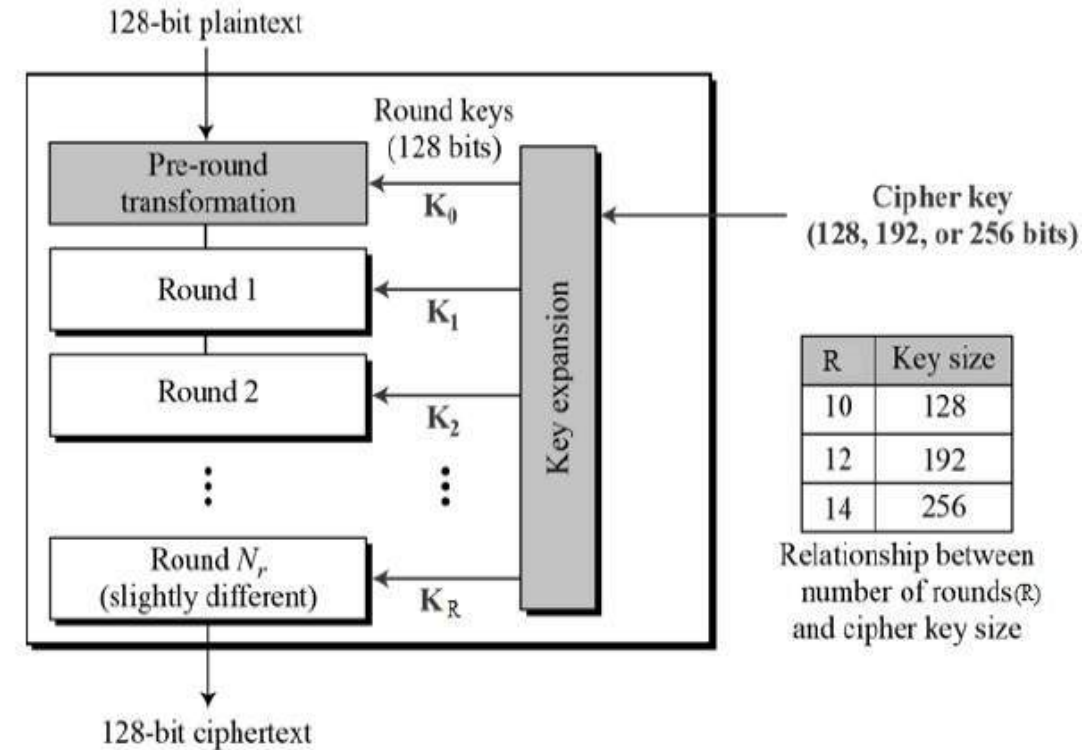
- The **DES** key length of 56 bits is insufficient for many modern applications
- Hence, DES can be used using **three keys** and **three iterations** of the main cipher
- Such a version is called **triple DES**, or **3DES**



- NIST deprecated DES and 3DES for new applications from 2017, and for all applications by the end of 2023
- Some organization still employ 3DES for compatibility reasons with legacy systems

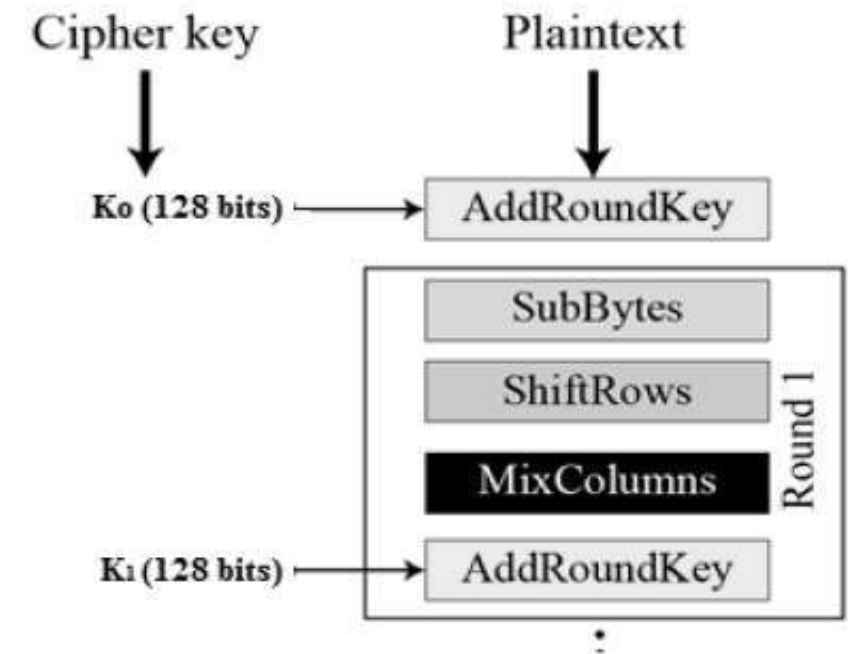
Symmetric Encryption – AES

- **Advanced Encryption Standard (AES)**
- Design performed publicly
- In fall 2000, the cipher proposed by Joan Daemen and Vincent Rijment was announced to be chosen as next encryption standard
- **AES** does not rely on Feistel cipher
- It is designed as a **Substitution-Permutation Network**
- The number of round is variable
 - It depends on the length of the key (128, 192, or 256 bits)
- Unlike DES, the encryption and decryption operation are distinct
 - Do not just require a re-ordering of the round keys
 - Sub-processes in each round are in reverse manner

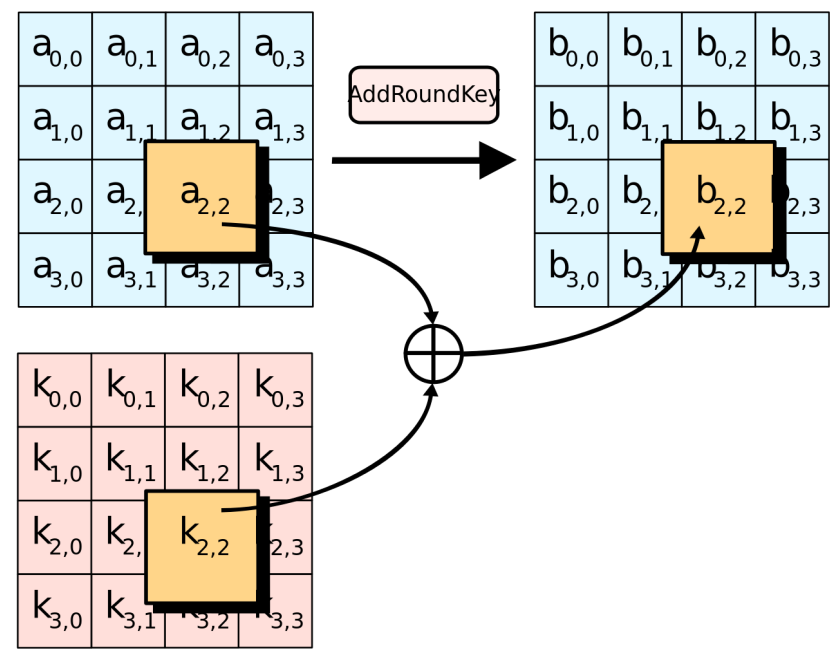
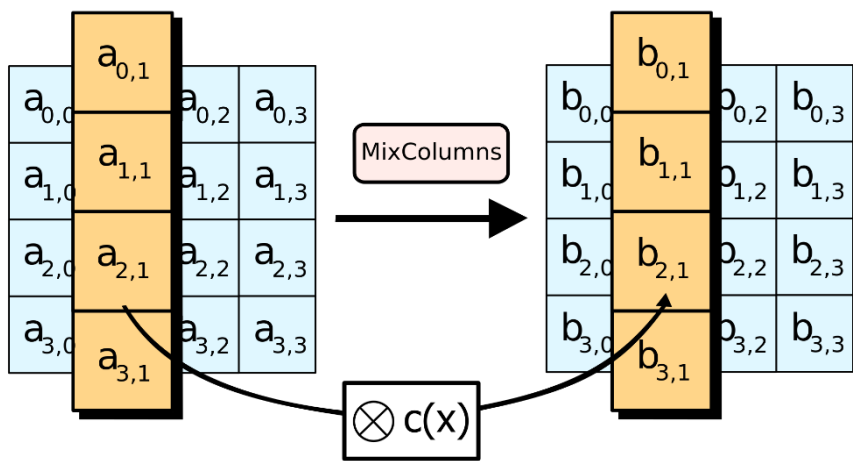
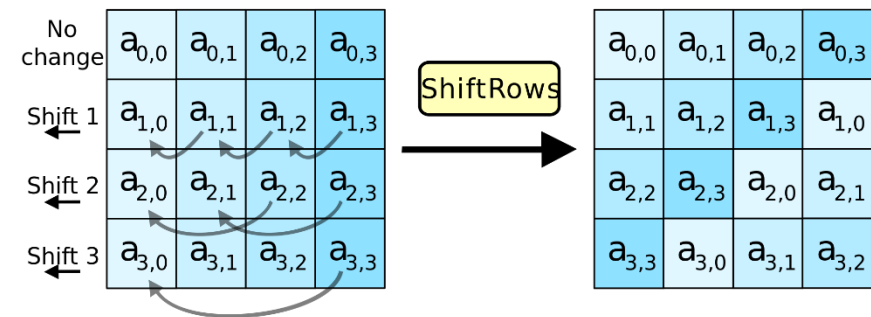
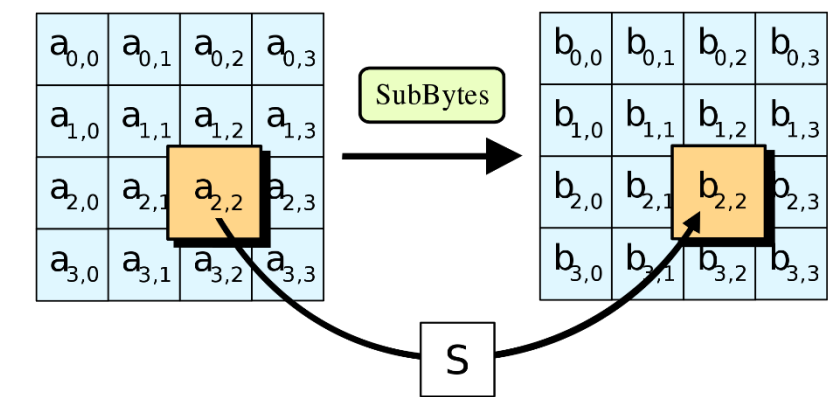


Symmetric Encryption – AES

- Each round comprise of four sub-processes:
 - **Byte Substitution (SubBytes)**
 - The 16 input bytes are substituted by looking up a fixed table (S-box) given in design
 - The result is a matrix 4 x 4, each cell containing one byte of data
 - **Shift Rows (ShiftRows)**
 - Each row of the matrix is shifted j times to the left. ($j = \text{row number}$)
 - Any entries that “fall off” is re-inserted on the right side of a row
 - **Mix Columns (MixColumns)**
 - Each column is transformed using a special mathematical function
 - This function takes in input the four bytes of one columns and outputs four completely new bytes, which replace the original column
 - **Add Round Key (AddRoundKey)**
 - The 16 bytes of the matrix are now considered as 128 bits and are XORed to the 128 bits of the round key
 - If this is the last round, then the output is the ciphertext
 - Otherwise, the resulting 128 bits are interpreted as 16 bytes, and a new round begins



Symmetric Encryption – AES



Symmetric Encryption – AES

- **Decryption Process**

- It is similar to the encryption process but in the **reverse order**
 - Add Round Key
 - Mix Columns
 - Shift Rows
 - Byte Substitution

- **Final remarks**

- AES is widely adopted and supported in both hardware and software
- No practical cryptanalytic attacks against AES have been discovered (yet)
- AES built-in flexibility of key length
 - Allows for a degree of “future-proofing” against progress in the ability to perform exhaustive key searches

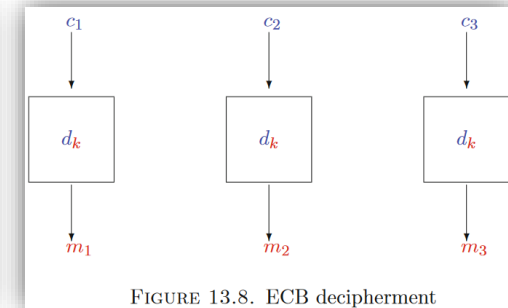
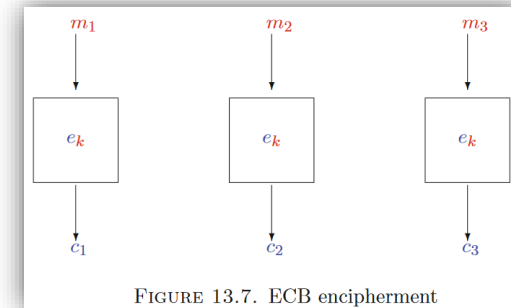
Symmetric Encryption – Modes of Operation

- Despite its name, a block cipher **is not** an encryption scheme
 - It is a building block to create an encryption scheme
 - It must be combined with a **mode of operation**
 - *A mode of operation refers to the technique used to repeatedly apply a symmetric encryption algorithm to encrypt a message longer than the algorithm's block size*
- The key advantage of this division is that we can design our **modes** and our **block cipher independently**
- Design goals:
 - For a **block cipher** is that it is a secure pseudo-random permutation
 - For a **mode of operation** is one of the following security goals:
 - One Way- (OW) or Indistinguishability of encryption (IDS-):
 - PASSive Attack (PASS)
 - Chosen Plaintext Attack (CPA)
 - Chosen Ciphertext Attack (CCA)
 - A designer of a mode of operation tries to prove mathematically that the mode satisfies the required security definition on the assumption that the block cipher is a secure Pseudo-Random Permutation (PRP)

Symmetric Encryption – Modes of operation

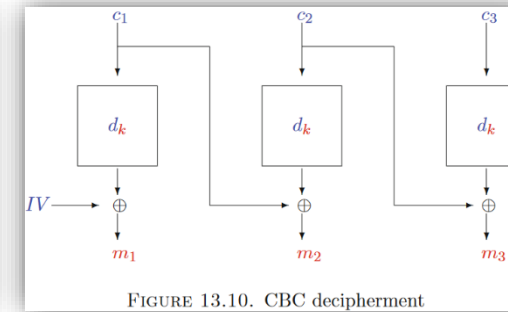
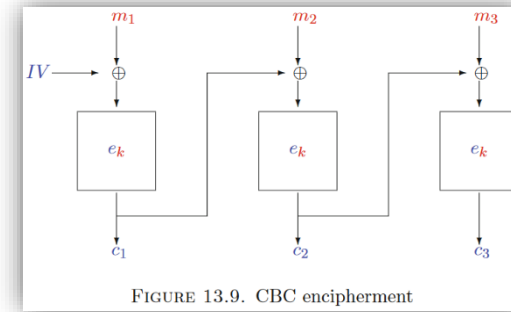
- **ECB (Electronic Code Block)**

- Simple to use
- Suffers from possible deletion and insertion attacks
- A one-bit error in the ciphertext give a one whole block error in the decrypted plaintext



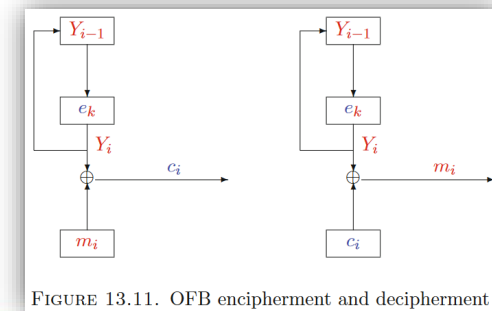
- **CBC (Cipher Block Chaining)**

- Protect against deletion and insertion attacks
- A one-bit error in the ciphertext gives only a one-block error in the corresponding plaintext, but also a one-bit error in the next decrypted plaintext block



- **OFB (Output Feedback)**

- Turns a block cipher into a stream cipher
- A one-bit error in the ciphertext gives a one-bit error in the decrypted plaintext



Symmetric Encryption – Modes of operation

- CFB (Cipher Feedback)
 - Turn a block cipher into a stream cipher
 - A one-bit error in the ciphertext affects both this block and the next, just as in CBC mode
- CTR (Counter Mode)
 - Turns block cipher into stream cipher, but enables blocks to be processed in parallel, thus providing performance advantages when parallel processing is available

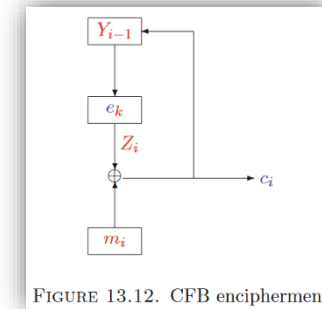


FIGURE 13.12. CFB encipherment

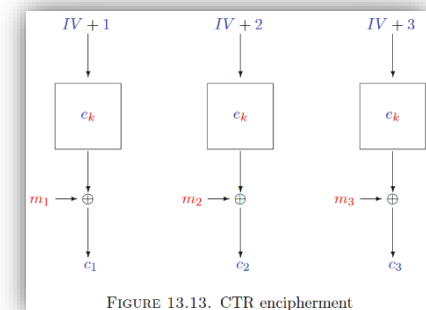


FIGURE 13.13. CTR encipherment

Symmetric Encryption – Modes of Operation

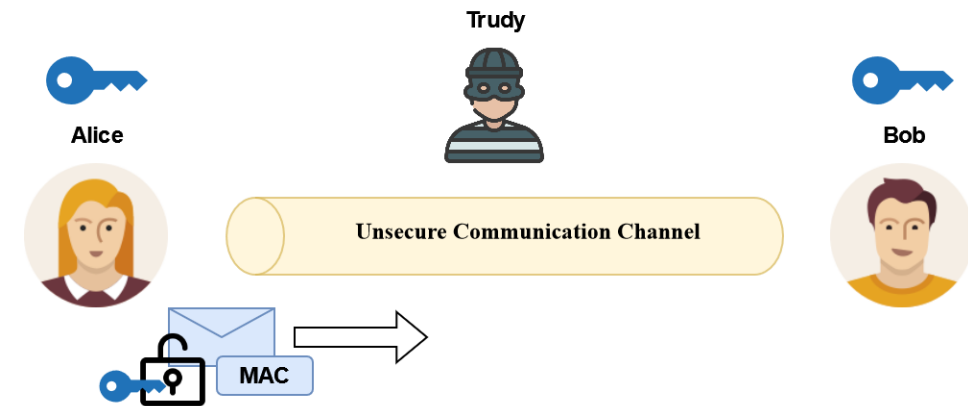
Mode	IV	IND-PASS	IND-CPA	IND-CCA
ECB Mode	-	Thm 13.2 $\Rightarrow$ ✗	$(\neg \text{IND-PASS}) \Rightarrow$ ✗	$(\neg \text{IND-PASS}) \Rightarrow$ ✗
CBC Mode	fixed nonce random	(nonce IND-PASS) $\Rightarrow$ ✓ (random IND-PASS) $\Rightarrow$ ✓ IND-CPA $\Rightarrow$ ✓	Trivially ✗ Thm 13.5 $\Rightarrow$ ✗ Thm 13.6 $\Rightarrow$ ✓	Trivially ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗
OFB Mode	fixed nonce random	(nonce IND-PASS) $\Rightarrow$ ✓ (random IND-PASS) $\Rightarrow$ ✓ IND-CPA $\Rightarrow$ ✓	Trivially ✗ $(\neg \text{OW-CPA}) \Rightarrow$ ✗ Prf of Thm 13.6 $\Rightarrow$ ✓	Trivially ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗
CFB Mode	fixed nonce random	(nonce IND-PASS) $\Rightarrow$ ✓ (random IND-PASS) $\Rightarrow$ ✓ IND-CPA $\Rightarrow$ ✓	Trivially ✗ Thm 13.10 $\Rightarrow$ ✗ Prf of Thm 13.6 $\Rightarrow$ ✓	Trivially ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗
CTR Mode	fixed nonce random	(nonce IND-PASS) $\Rightarrow$ ✓ IND-CPA $\Rightarrow$ ✓ IND-CPA $\Rightarrow$ ✓	Trivially ✗ Thm 13.11 $\Rightarrow$ ✓ Thm 13.11 $\Rightarrow$ ✓	Trivially ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗ $(\neg \text{OW-CCA}) \Rightarrow$ ✗

TABLE 13.2. Indistinguishability security properties of the five basic modes of operation

Symmetric Encryption – Modes of Operation

- All the prior modes of operation did not provide security against **Chosen Ciphertext Attacks (CCA)**
- We need more advanced modes of operation, called **Authenticated Encryption Modes**
- Modes which provide this property for encryption based on block cipher include:
 - Chaining-message authentication Code Mode (CCM)
 - Galois/Counter Mode (GCM)
 - Offset Codebook mode (OCB)
- **Encrypt-then-MAC**
 - Standard method of adding integrity protection to the ciphertext is to append a **Message Authentication Code (MAC)** to the ciphertext.

$$\begin{aligned}c_1 &\leftarrow E(k_1, m) \\c_2 &= MAC(k_2, c_1) \\c &= (c_1, c_2) \\v &\leftarrow Verify(k_2, c_1, c_2) \\m &\leftarrow D(k_1, c_1)\end{aligned}$$



Message Authentication Code

- Symmetric encryption provides **data confidentiality**
- **BUT** it doesn't provide **data integrity** nor **authentication**
 - Ciphertext can be modified either by communication errors or an attacker
- **Solution:** append a **Message Authentication Code (MAC)** to the encrypted message

$$MAC(k, m) = H(k \oplus m)$$

- Where:
 - **H** is a cryptographic hash function (🤔? We will soon understand what a cryptographic hash function is!)
 - **m** is the message to be authenticated
 - **k** is the secret key
- **HMAC**
 - Known as **Keyed-hash message authentication code** or **hash-based message authentication code**.

$$HMAC_k(m) = H((k \oplus opad) || H((k \oplus ipad) || m))$$

Message Authentication Code

- **Limitation:**
 - Inability to provide **non-repudiation**
 - Non-repudiation is the assurance that a message originator cannot deny any previously sent messages and commitments or actions
 - **MAC** techniques **do not** provide a non-repudiation service. If the sender and the receiver get involved in a dispute over a message's originator, **MAC cannot provide a proof that a message was indeed sent by the sender**
 - Though no third party can compute the MAC, still sender could deny having sent the message and claim the receiver forged it
 - **It is impossible to determine which of the two parties computed the MAC**
- This limitation can be overcome by using **public key based digital signatures!**

Hashing

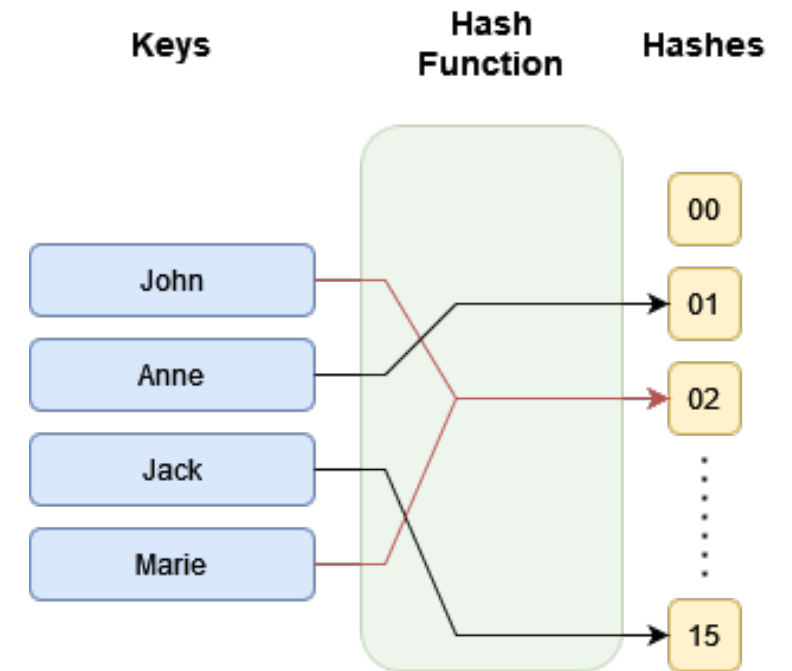
- Hash Function

- A **Hash Function H** is a function which takes an **arbitrary length bit string as input** and produces a **fixed-length bit string as output**

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^l$$

$$H(x) = y$$

- The output is often called a **digest**, **hash code**, or **hash value**



Hashing

- **Cryptographic Hash Function**

$$H(x) = y$$

- It **must** satisfy the following three properties:

- **Preimage Resistant** (One-wayness):

- Given a hash value, it should be hard to find a message which generated it
 - Given a digest y , it is computationally infeasible to find a message x , such that $H(x) = y$

- **Second Preimage Resistant** (Weak Collision Resistance):

- Given one message, it should be hard to find another message with the same hash value
 - Given a message x , it is computationally infeasible to find another message x' such that $x \neq x'$ and $H(x) = H(x')$

- **Collision Resistant** (Strong Collision Resistance):

- It should be hard to find two messages with the same hash value
 - It is computationally infeasible to find two messages x and x' such that $x \neq x'$ and $H(x) = H(x')$

Hashing

Hash Function	Year	Output Size	Security	Hash Example (CRPG)
MD5	1991	128	Weak	c780a74a76595794aaf388d46f70d60a
SHA1	1995	160	Weak	5da3f5de38f2eef9e3806122fb0583c0040b09c4
SHA2	2001	224, 256 , 384, 512	Moderate/Strong	1d0ea483bafea590b11a2f7e446e35486b14dbaef9cabd3a64bbdea135f82e14
SHA3	2015	256 , 384, 512	Strong	fedc8aee4605952b51d7567c71d712f07816ed6de6e351f8268ff2758502007a
BLAKE3	2020	256 , 384, 512	Strong	98955d201c6cc37b5e8eba5c9d6c94b90757012a256ec5838523b494bedf747a
...	...	...	...	...

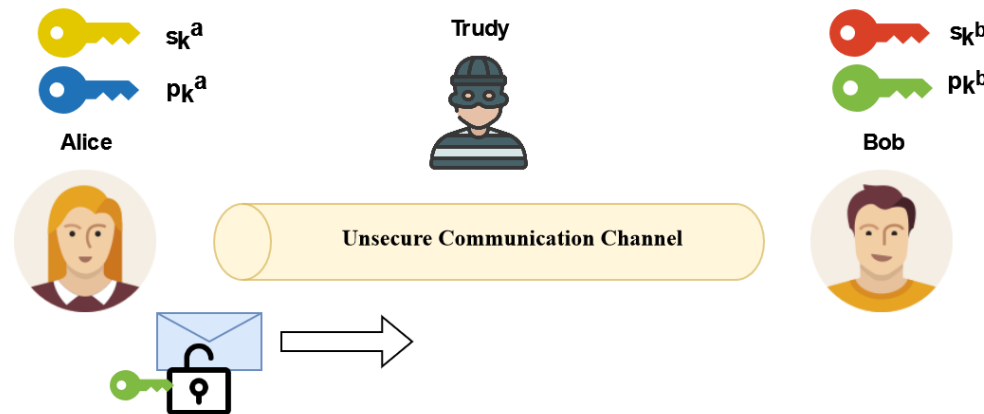
Hashing

- Hash Function use cases:
 - File integrity
 - **Message Authentication Code**
 - **Digital Signatures**
 - Passwords storage
 - Blockchain
 - Data retrieval
 - Caching

Asymmetric Encryption

- Two **different** keys are used for plaintext encryption and ciphertext decryption
 - Each user must possess a pair of keys to send encrypted messages and decrypt them
 - The sender uses the receiver **public key** (p_k) to encrypt the message and then transmits it to the receiver
 - The receiver uses its own **private key** (s_k) to decrypt the received message

$$d(s_k^b, e(p_k^b, m)) = m$$



- Public and private keys are mathematically linked, but **from pk it's not possible to retrieve sk**

Asymmetric Encryption - RSA

- RSA – Ron Rivest, Adi Shamir, and Len Adleman
- World's first public key encryption algorithm (1977).
- Based on the difficulty of the **RSA problem**:
 - **Difficulty of finding the prime factors of large integers**

$$n = p \cdot q$$

- Usually, ***n*** is a 512-bits long number
- Algorithm steps:
 - Key Generation
 - Encryption
 - Decryption

Asymmetric Encryption - RSA

• Key Generation

➤ Generate the RSA modulus n

- Choose two large prime numbers p and q
- Calculate n

$$n = p \cdot q$$

➤ Find derived number e

- e must be greater than 1 and less than $(p-1)(q-1)$
- There must be no common factor for e and $(p-1)(q-1)$ except 1

$$\gcd(e, (p-1)(q-1)) = 1$$

➤ From the **public key**

- The pair of numbers (n, e) form the RSA public key and is made public

$$p_k = (n, e)$$

➤ Generate **private key**

- Private key d is calculated from p , q , and e
- For given n and e , there is unique number d

$$d \cdot e = 1 \pmod{(p-1)(q-1)}$$
$$s_k = (d, p, q)$$

Asymmetric Encryption - RSA

- Encryption

- Suppose the sender wish to send some text message to someone whose public key is (n, e)
- The sender then represents the plaintext as a series of numbers less than n
- To encrypt the first plaintext P , which is a number modulo n , the encryption process is:

$$C = P^e \bmod(n)$$

- Decryption

- To decrypt the ciphertext C , the decryption process is:

$$P = C^d \bmod(n)$$

Asymmetric Encryption - RSA

- A numerical example:

- Let two primes be $p = 7$ and $q = 13$
- Thus, modulus $n = p \cdot q = 7 \cdot 13 = 91$
- Select $e = 5$, which is valid choice since there is no number that is common factor of 5 and $(p-1)(q-1) = 6 \cdot 12 = 72$, except for 1
- The pair of numbers $(n, e) = (91, 5)$ forms the **public key** and can be made available to anyone
- Input $p = 7$, $q = 13$, and $e = 5$ to the Extended Euclidean Algorithm. The output will be $d=29$
- The triple of numbers $(d, p, q) = (29, 7, 13)$ forms the **private key** and should be kept secret
- Encryption: with plaintext $P = 10$, we get ciphertext $C = 10^5 \bmod 91 = 82$
- Decryption: $P = 82^{29} \bmod 91 = 10$

Asymmetric Encryption - RSA

- **Security considerations:**

- **“Raw” RSA** function does not provide a semantically secure encryption scheme

- To make it secure we need to:

- Add redundancy to the plaintext before encryption, or
- Add redundancy to the ciphertext, and
- Use a random padding scheme for the plaintext.

- **Optimized Asymmetric Encryption Padding (OAEP):**

- Add randomness and structure to plaintext before encryption
- **RSA-OAEP**

Asymmetric Encryption - ECC

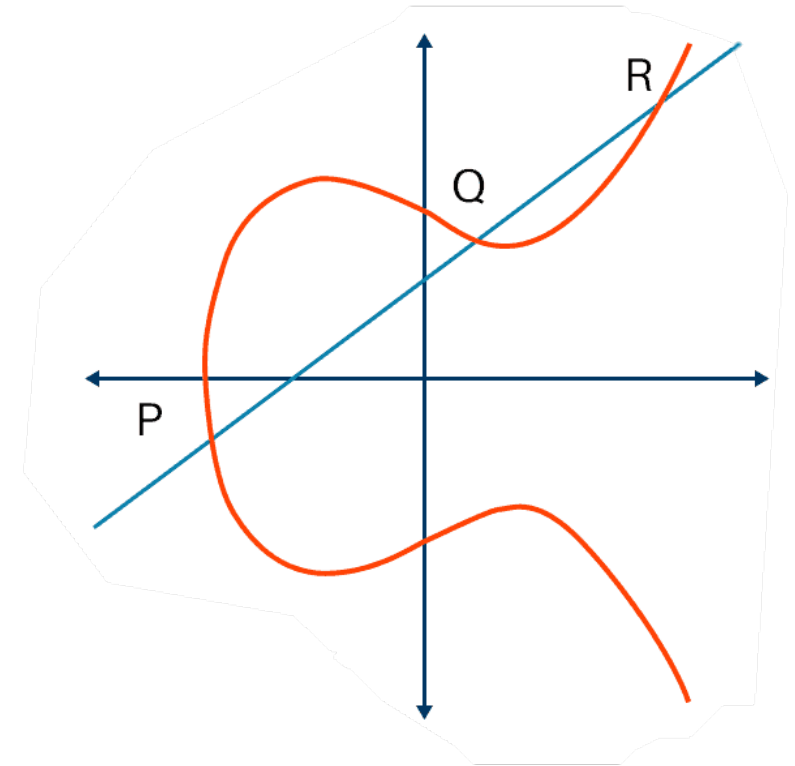
- Elliptic Curves Cryptography

- Defined over a field K , and describes points in K^2
- An **Elliptic Curve (EC)** can be described by the equation:

$$y^2 = x^3 + ax + b$$

- Implementing **Group Operations**:

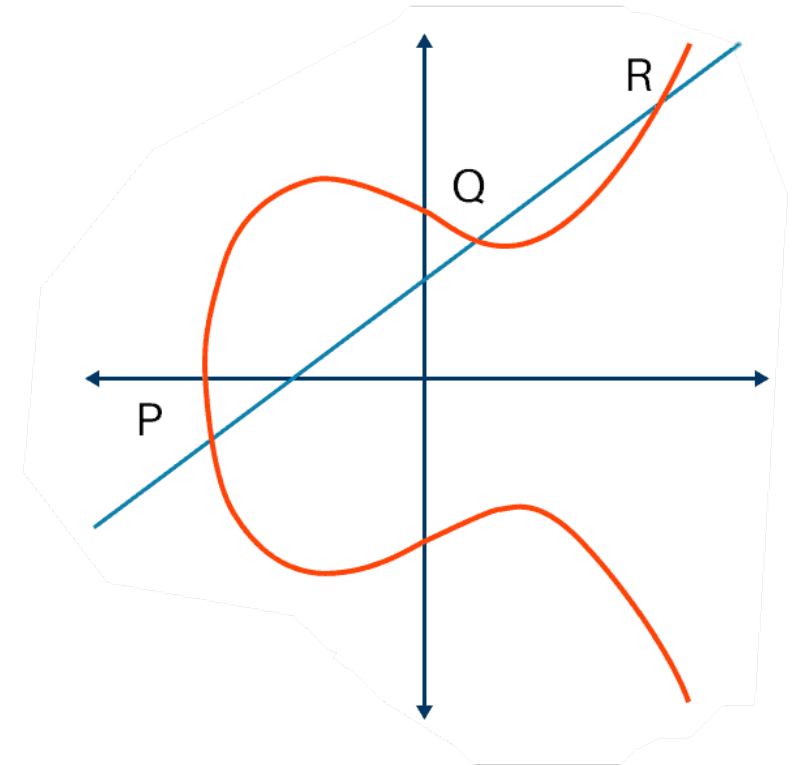
- Main operations: point addition and point multiplication
- Adding two points that lie on an **EC** results in a third point on the curve
- Point multiplication is repeated addition
- If P is a known point on the curve and it is multiplied by a scalar k , $Q=kP$ is the operation of adding $P + P \dots + P$ (k times)
- Q is the resulting public key, and k is the private key



Asymmetric Encryption - ECC

- Discrete Log Problem

- Difficult to find k given Q and P
- This is termed as the **discrete log problem**
- Methods to solve this problem include:
 - Brute force
 - Pollard's Rho Attack
- Both are computationally expensive and unfeasible



- The version applicable to ECC is called **Elliptic Curve Discrete Log Problem**

Asymmetric Encryption

- RSA vs Elliptic Curves

Security (in bits)	RSA Key Length	ECC Key Length
80	1024	160-223
112	2048	224-255
128	3072	256-383
192	7680	384-511
256	15360	512+

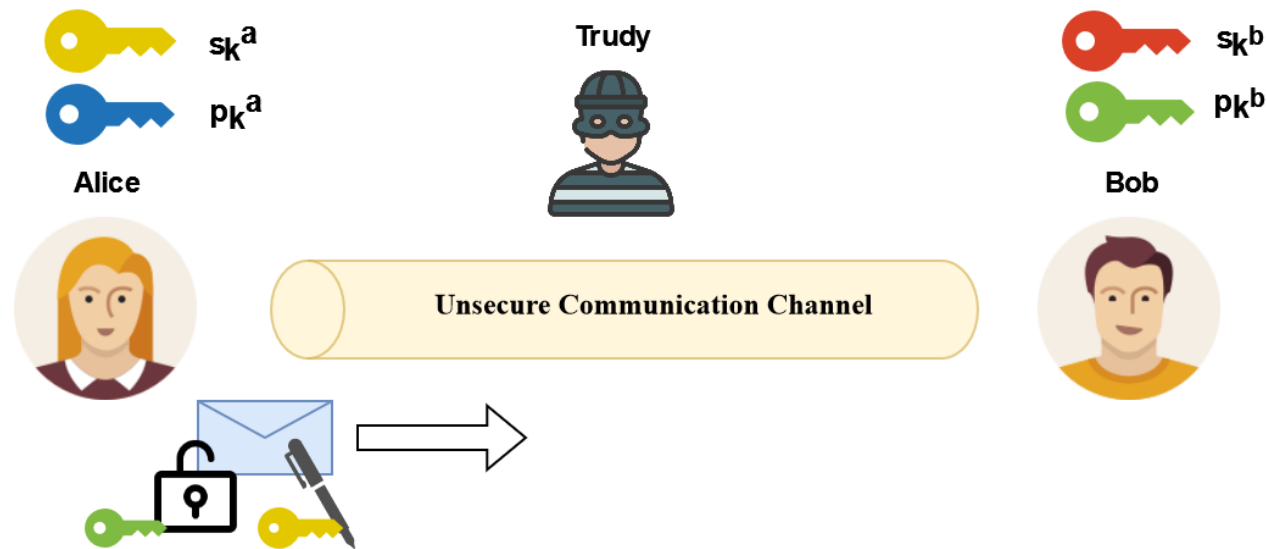


Both RSA and ECC are quantum-unsecure!

RSA	ECC
Well-established	Newer
Prime factorization method	Elliptic curves
Faster	Slower
Heading towards the end of its tenure	Adaptive phase Usage scaling up
Longer keys	Shorter keys

Digital Signatures

- **Digital Signatures** are public-key primitives of message authentication
- Bind a person/entity to the digital data
 - This binding can be independently verified by receiver as well as any third party
- Calculated from the **data** and a **secret key** known only by the signer



Digital Signatures

- **Digital Signatures** provide:
 - **Message Authentication**
 - ✓ Receiver can verify that the message has been not been tampered with
 - **Data Integrity**
 - ✓ Receiver can verify the integrity of the data
 - **Non-repudiation**
 - ✓ Receiver can prove that the sender sent that message
- Two possibilities:
 - **Sign-then-Encrypt**
 - **Encrypt-then-Sign**



To be preferred!

Digital Signatures

- **Digital Signatures** algorithms:
 - RSA – Full Domain Hash (RSA-FDH)
 - **RSA – Probabilistic Signature Scheme (RSA-PSS)**
 - Digital Signature Algorithm (DSA)
 - **Elliptic Curve Digital Signature Algorithm (ECDSA)**

Thank You



GET IN
TOUCH

Alex Stefanov

Associate Professor, Chartered Engineer (CEng MIEI)

Email: A.I.Stefanov@tudelft.nl

Cyber Resilient Power Grids ([LinkedIn](#))

Control Room of the Future ([LinkedIn](#))

Intelligent Electrical Power Grids, EEMCS, **TU Delft**
Mekelweg 4, 2628 CD Delft, The Netherlands